DeepL
订阅DeepL Pro以翻译大型文件。
欲了解更多信息，请访问www.DeepL.com/pro。

AVL和红-黑树的性能比较评估

Svetlana Štrbac-Savić
电气工程和计算机科学学院应用研究
塞尔维亚贝尔格莱德，Vojvode Stepe 283
+381-11-3950026
svetlanas@viser.edu.rs

Milo Tomašević
塞尔维亚贝尔格莱德Bulevar Kralja
Aleksandra 73号电气工程学院
+381-11-3218392
mvt@etf.rs

ABSTRACT
AVL和红黑树是二元搜索树的次优变体，可以实现搜索操作的对数性能，而不需要付出过多的最优平衡成本。在介绍了简要的理论背景后，本文对这两种结构的性能进行了比较性评价。评估是通过合成工作负载模型的模拟来进行的。为了获 得 更好的洞察力，性能指标被选择为与实施和平台无关。给出并讨论了一些有代表性的评估结果。最后，本研究的结果被总结为对所分析的树的最佳使用建议。

类别和主题描述符
E.1 [**数据结构**]：*树*；F2.2 [**算法和问题复杂性的分析**]：非数字算法和问题--*排序和搜索*

一般条款
算法, 性能

关键词
二进制搜索树、AVL树、红黑树

1. 简介
二进制搜索树（BST）代表了一种基本结构，旨在有效地搜索和维护主存储器中的动态数据集[1]。二进制搜索树的优势来自于其动态结构，确保了有效的插入和删除。一个更重要的特点是，BST保证了搜索、插入和删除操作在最佳和平均情况下的对数复杂性。为了避免最坏情况下的拓扑结构并实现最佳搜索性能，树形平衡技术是必不可少的。然而，由于每个节点插入和删除后的完美平衡可能非常昂贵（在最坏的情况下甚至达到$O(log\ n)$），所以它经常被诉诸于次优的

允许为个人或课堂使用本作品的全部或部分内容制作数字或硬拷贝，但不得以营利或商业利益为目的制作或分发拷贝，拷贝应在第一页注明本通知和完整的引文。以其他方式复制、再版、在服务器上发布或重新分配给名单，需要事先获得具体许可和/或付费。
BCI'12，2012年9月16-20日，诺维萨德，塞尔维亚。
Copyright 2012 ACM 978-1-4503-1240-0/12/09 ...$15.00.

该树的变体。他们放宽了平衡标准，平衡开销急剧下降。搜索操作复杂度的对数级，与最佳平衡结构中的相同，仍能保证有一个相当可接受的恒定退化系数。

基于不同的平衡技术（如有界平衡树中的权重平衡），已经提出了不同的结构，但最流行和最广泛使用的BST结构是基于高度平衡的。这类结构中最具代表性的是AVL树和红黑树。本文的主要目标是通过使用一个适当选择的合成工作负载模型，对这两种结构进行定性的性能比较评估。在第二部分，给出了AVL树的定义并讨论了一些特征，而第三部分阐述了红黑树的相同问题。在第四节中，描述了评估方法和工作负载模型。在第五节中，提出并讨论了比较评估的一些定量结果。

2. AVL树
让我们把一个节点b的*平衡定义*为其左、右子树的高度之差（分别为hl和hr）。然后，AVL树被定义为二进制搜索树，其中每个节点的平衡的绝对值最多为1[2]。空树的高度被定义为0。

这样一来，平衡标准就大大放宽了。例如，与最佳平衡树的叶子只能部署在两个最低层不同，在一个极端情况下，AVL树的叶子可以跨越beetwen层h和$2h$*的*范围。在给定的节点数下，具有最大高度的AVL树的最差拓扑结构被称为Fibonacci树。

在AVL树中，节点平衡只能是1、0和-1。余额为1的节点向左倾斜，余额为-1的节点向右倾斜。一些插入或删除操作会扰乱一些节点的平衡，但是，只要它们在允许的范围内，就不需要平衡树。然而，当至少有一个节点的余额变成2或-2时，就会进行一些树的调整操作（称为旋转），以使所有节点的余额恢复到允许的范围内。旋转的成本相对较低，频率也不高，所以维护AVL树的开销是可以接受的[1]。

尽管AVL树只是 "接近 "平衡，但操作的时间复杂度是$O(log\ n)$，其中n是树上的节点数。在[3]中证明，对于有n*个*节点的AVL树，其高度h满足以下条件

$$h < 1.4405 \log_2 (n+2) - 0.327。$$

因此，AVL树中的搜索、插入和删除操作，以及寻找最小、最大、前任和后任等操作的复杂度为*O(log n)*，因为它们在高度为*h*的树中的复杂度为*O(h)*，而每个内部有*n*个节点的AVL是高度为*O(log n)的*二进制搜索树。

3. 红黑色的树

红黑树，就像AVL树一样，是一种近乎平衡的树，其中每个节点和分支都以其颜色为特征。着色规则有严格的定义。

如果二进制搜索树满足以下条件，那么它就是一棵红黑树：

1. 每个节点不是红色就是黑色。
2. 每一片叶子（NIL）都是黑色的。
3. 如果一个节点是红色，那么它的两个儿子都是黑色。
4. 从根到叶的每条路径都包含相同数量的黑色节点。[4]

在本文的其余部分，为了简洁起见，这些树将被称为RB树。

RB树被用作2-3-4树在二叉树形式下的实现[8]。2-3-4树的一个重要特征是，所有的叶子都在同一水平线上，这保证了最佳的平衡。除了二进制树中常见的具有一个键和两个子树的2个节点外，这些树还可以有具有两个键和三个子树的3个节点，以及具有三个键和四个子树的4个节点。2-3-4树是4度的B树，与RB树同构[9]，这意味着对于每个2-3-4树，至少有一个RB树作为其二进制表示[5]。这个事实很重要，因为2-3-4树的实现相当复杂，因为在操作中，必要时，一种类型的节点必须转化为另一种。这些转换的要求很高，可能会影响性能。此外，树的度数越高，使用的内存就越多。因此，它以标准二叉树的形式表示是相当有吸引力的。

通过RB树实现2-3-4树的想法包括用红色分支连接的小二叉树表示3节点和4节点，而黑色分支连接2-3-4树的原始节点。与父母有黑色分支的节点是黑色节点，而与父母有红色分支的节点是红色节点。

2-3-4树的2、3、4节点的映射按以下方式进行：

- 2-节点被表示为有两个黑色儿子的二进制节点。
- 3节点表示为两个二进制节点，一个是红色，另一个是黑色，因此节点之间的分支是红色。3-节点可以用两种方向表示，这取决于平衡要求。
- 4-节点表示为三个二进制节点的集合，黑色节点为父节点，而子节点为红色。使用两个红色分支。

这种节点映射的结果，以及所有叶子都在同一高度的事实，保证了RB定义得到满足。它还确保了树的宽松平衡，这又减少了树维护的开销。

如果树的结构被一些插入或删除的操作所修改，从而损害了定义的要求，则以旋转的形式进行树的转换，以重新建立正确的结构和着色。RB树的基本操作已经在[5]、[6]和[7]中描述过。就像AVL树一样，这些树的对数性能也得到了保证，这一点在[4]中得到了证明。事实证明，具有*n个*内部节点的RB树的高度不超过$2\cdot \log_2 (n+1)$。因此，由于高度为*h*的*n个*节点的RB树中的通常操作是*O(h)*级的，因此，它的时间复杂度是*O(log n)*。

4. 相关的工作

在公开的文献中可以找到一些关于二元搜索树与宽松的平衡准则的比较研究。Carlton研究了HB[k]树的性能，即高度平衡树，其中AVL树是*k=1*的一个特例。评估HB[k]树的性能的模拟结果在[10]中提出。结论是，对于k>1，维护HB[k]树的程序的执行时间与树的大小无关，除了在删除操作中为了恢复HB[k]特性而重新访问的节点的平均数量。随着允许的不平衡度（k）的增加，维护HB[k]树的成本明显下降。讨论了显示维护HB[k]树的成本与k的关系的分析和实验结果。就AVL树而言，只有搜索时间是树大小的函数，在一般情况下，维护不取决于树的大小。在插入的情况下，平均需要0.465次旋转操作，为了调整高度，需要重新访问2.78个节点。另外，每次删除操作平均需要0.214次旋转，维护高度属性时重新访问的节点的平均数量为1.91。

Bear和Schwab在[11]中通过对合成工作负载的模拟，经验性地比较了平衡树和权重平衡树的高度。在结论中，他们优先选择了AVL树。

Wright在[12]中对九种不同类型的树进行了比较分析，如AVL树、几种类型的权重平衡的树。

树（有限平衡的树，系数α=1-√2，在[13]中描述），搜索的节点向根部移动一级的树[14]，以及一些算法的适当组合。评估衡量了不同类型的输入序列的执行时间。考虑的操作是插入和搜索，而删除没有考虑在内。结论是，最有效的树是一种由某些操作增强的AVL树。

在[15]中提出的研究可能是最接近本文的概念，其中考虑并比较了一些类型的AVL树和几种类型的自调整二进制搜索树（[16]）[16]。结论是AVL树是最有效的，而在自调整结构中，自上而下的拼合操作时，拼合树给出了最好的结果[17][18]。

在[19]中还比较了三种类型的二进制搜索树：AVL、RB和splay树。比较了这些树在系统软件中应用时的性能，因此对各种平台进行了测试。由于在测试中考虑了几种插入操作，RB树对随机数据集更有效，AVL树在排序数据集中更好。当涉及到搜索操作时，AVL树也更有效率。

AVL和RB树也在[20]中进行了比较。除了这些类型的树之外，跳过列表和哈希表也被评估了。这种比较是基于字典实现中所述结构的效率。作者优先考虑了RB树。

尽管[15]和[19]的研究与本文的主题相当接近，但不管分析的树的选择是否相同，都是从不同的角度进行比较的。以前的一些研究（例如，[19]）观察了树木应用的特定环境，而我们的研究则提出了

更普遍的性能研究，独立于算法的实施、操作系统、具体应用和进行测试的机器。

5. 评价方法

AVL和RB树的比较性能评估是通过特定的合成工作负载的模拟方法进行的。工作负载的特点是相当多样的，涉及的参数包括：键的数量、键值的范围、键值的分布、时间定位、搜索、插入和删除操作的相对频率、搜索成功和不成功的概率等。性能指标的选择是独立于算法的实施和测量的平台的。

仿真环境以及树形操作本身的算法都是用C语言编程，Microsoft Visual Studio 6.0实现的。所有的实验都是在WindowsXP平台上进行的，版本为2002，使用奔腾4，3.00GHz（英特尔奔腾4），1GB内存。

为了更全面地了解AVL和RB树的性能，产生了密钥序列。取出钥匙值的时间间隔、序列中的元素数量和钥匙值的频率是不同的，因此要注意模拟钥匙的时间定位。

密钥序列的长度被选择为10到1000000个元素，以10的倍数计算。序列中的元素数量是不同的，以便确定性能如何取决于钥匙的数量。

关键序列中的数值可以重复。从中提取关键值的区间和关键序列的大小是不同的，据此，形成的序列的元素数是六个所述值中的一个。区间的下限从零开始，而上限则因情况不同而不同。这里使用了两组密钥序列，它们根据密钥生成的方式而不同。

在第一组中，钥匙是通过*rand*方法产生的，即从一定的区间中选择元素，这些元素是

变化，而且序列元素的数量也在变化。用于存储这些序列的文件名表明了随机密钥的分布情况（名称以R开头，间隔为下上限，以及序列中的键的数量。例如，R1000_100000.txt是一个包含100000个元素的序列的文件，其随机选择的键值来自区间[0，999]。序列中键值的区间上限属于{10, 100, 1000, 10000, 30000, 40000, 100000, 1000000}这个集合。这样的键值选择的后果是，一些来自区间的值可以重复出现，而一些值不出现在序列中。

在第二组中，键值也可以在一个序列中重复，序列元素的数量也是不同的，键值的间隔也是不同的，其中数量和间隔的值与第一组相同。主要的区别是在序列中下一个键的生成方法。形成这个钥匙序列子组的目的是为了获得非均匀分布的钥匙序列，并表达所选值的时间定位，这有时是树状访问的特征。该函数

$$y = \frac{1 - x}{1 + ax} \qquad (1)$$

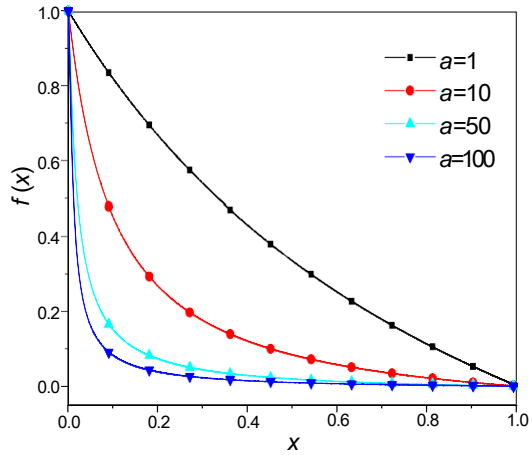
是以下列方式来执行不同程度的时间定位的。形成一个没有重复值的辅助初始序列（以下称序列*密钥*）。然后，从区间$x \in [0,1]$中随机生成x。

有了这样的x，使用函数（1）y被计算出来，可以注意到，对于$a>0$，$y \in [0, 1]$。如果*密钥*序列的大小为

用n表示，索引i现在被计算为$i=n \cdot y$。最后，索引为*i的密钥*序列的元素被输入到新生成的序列中。这个过程重复进行，直到

产生了所需长度的序列。

通过改变参数a，可以调整曲线的凹陷性，如图1所示。当参数a的值越高，所产生的序列中的键的时间定位水平就越高。参数a取值为1、10、50和100，并分析了a=100的结果。序列的大小以及区间的上限属于{10, 100, 1000, 10000, 100000, 1000000}集。结果序列被存储到文件中，其名称以下列方式形成：



和

图1：参数*a*的不同值区

间上的函数（1）值。 $x\in\left[0,1\right]$**为**

nrIntervalUpperLimit SequenceElementsNumber，例如。

nr10000_100000.txt。

这样的合成工作负载模型允许评估键的数量如何影响对所选树型的一系列操作的效率，以及键的重复和键值分布如何影响有关每个树型的性能。

我们考虑了一系列三类操作的测量结果：搜索、插入和删除键值。搜索、插入和删除操作在操作系列中的百分比是不同的，它们的相对顺序是随机确定的。百分比的选择是为了使搜索是最频繁的，而插入和的频率是相同的。这种选择是基于这样一个事实，即搜索是实际条件下最普遍的操作。

对不同的初始树进行了一系列的树操作。本文介绍了用随机插入的区间[0 ... 99999]中的数值生成的初始树得到的结果。在讨论一系列三类操作的结果时，没有考虑形成初始树的成本，但它们的成本和选择将在下一节开始讨论。收集了以下性能指标：

- 搜索过程中的平均高度 - 在搜索成功的情况下找到元素的平均高度，或者在搜索不成功的情况下，搜索结束的树节点的高度。
- 树高 - 这个参数指的是进行搜索序列的初始树的最大高度。
- 每次操作的平均旋转次数。

6. 结果和讨论

为了设置场景，应该建立初始树，在此基础上进行一系列的操作。我们研究了两种建立初始树的方法：通过插入排序的键值和插入随机选择的值。图2显示了使用相同的键插入排序序列（I10到IMillion）和随机序列（R10到RMillion）产生的10到100万节点的AVL和RB树的高度。从图2中可以得出结论，两种树对于插入键的排序顺序都有较低的高度。在这种情况下，AVL树的高度比RB树低，而且它们的高度差随着插入的数值的增加而增大。如果键的插入顺序是随机的，那么差异就比较小。在大多数情况下，甚至两种树的高度都是相等的。通过插入排序后的键来建立初始树，会使后续的性能评估结果偏向于以搜索操作为主的一系列操作。在这种情况下，AVL树会有更好的性能；因为搜索树的操作不需要任何旋转，其效率主要取决于树的高度。另外，随机生成的初始树被认为是比从排序序列建立的树更典型的情况。

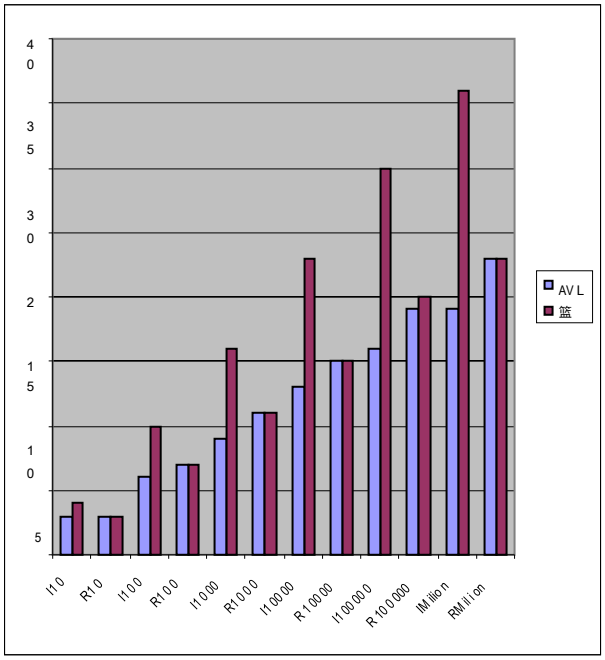


图2：用插入排序和随机排序的键建立的AVL和RB树的高度。

从表1中给出的结果可以看出，正如预期的那样，建立最初的AVL树比RB树需要更多的旋转。所以建立AVL树的成本较高，但在排序键插入的情况下是合理的。这种优势随着操作序列中搜索操作的数量而增加。

表1：在随机和排序的关键值中，不同数量的插入操作系列的旋转次数

值的数量	旋转的次数（插入随机值）。		旋转的次数（插入排序的值）。	
	AVL	RB	AVL	RB
10	2	2	6	5
100	68	68	93	89
1000	646	646	990	983
10000	6452	6452	9986	9976
100000	64372	64372	99983	99969
1000000	640287	640287	999980	999963

下面是对一系列操作的测量结果，其中有10%的插入，80%的搜索，10%的键删除。最初的树是由一串具有100,000个不同值的随机顺序的键建立的，没有重复。之所以采取初始树，是因为现实中已经有一些数据在内存中，因此删除和搜索操作在系列操作开始时不会有高比例的不成功。建立初始树的成本没有被考虑在内。初始AVL树的最大高度为19，而RB树的最大高度为20。关于这两种类型的树，在操作性能期间，树的最大高度与初始高度相比没有变化。操作的数量固定为100,000。

表2：在密钥序列中测量100,000个操作系列的结果（10%的插入、80%的搜索和10%的删除），其值是随机部署在序列中的。

文件名称	每次操作的平均高度		每个人的平均轮换次数操作	
	AVL	篮球场	AVL	篮板球
R10_100000.txt	14,459	15,729	0,052	0,040
R100_100000.txt	14,231	15,594	0,056	0,046
R1000_100000.txt	12,716	13,618	0,058	0,048
R10000_100000.txt	14,174	15,168	0,066	0,059
R100000_100000.txt	14,795	14,864	0,083	0,073
RMilion_100000.txt	17,101	17,660	0,165	0,136

表2数据显示了通过一系列操作获得的平均高度和每次操作的平均旋转次数的数据。一系列的基本操作是对随机分布的钥匙序列进行的，其中的钥匙可能会被重复。AVL树似乎更有效率。RB树每次操作的旋转次数较少，但差异小于高度的差异，因此，尽管旋转是一个比水平搜索更昂贵的操作（一个额外的递归调用或另一个重复的迭代检查程序），AVL树的效率更好。可以注意到，对于这两种类型的树，当涉及到高度时，搜索键范围的增加并不会导致执行操作时达到的平均高度的增加或减少。原因是在搜索过程中，也就是最频繁的操作，代表了删除和插入操作的第一阶段，树的拓扑结构没有被修改，所以它所到达的高度取决于初始树中的键的位置。在RMilion_100000.txt输入序列的情况下，两种情况下的平均高度差异最大。这是预期的，考虑到这里出现的不成功的搜索是大于树的最大值的键，在这种情况下，搜索在树的叶子上结束。就旋转而言，可以注意到两个树的每个操作的平均旋转次数随着搜索键的范围的增加而增加。

表3：对密钥序列进行100,000次操作（10%的插入，80%的搜索和10%的删除），密钥值的分布不均匀。

文件名称	每次操作的平均高度		每个人的平均轮换次数操作	
	AVL	篮球场	AVL	篮板球
nr10_100000.txt	15,057	15,127	0,023	0,019
nr100_100000.txt	15,025	15,176	0,018	0,010
nr1000_100000.txt	14,841	15,899	0,009	0,006
nr10000_100000.txt	15,395	16,447	0,014	0,013
nr100000_100000.txt	15,265	15,292	0,023	0,021
nrMilion_100000.txt	16,961	17,168	0,060	0,050

表3显示了与表1相同的操作数量和部署的实验结果，但采用了非统一的密钥分布，强制执行了时间上的定位。在这种情况下、

AVL树比RB树更有效，同样。当涉及到这两个参数在价值范围增加的功能时，可以得出与前面情况相同的结论。表2和表3的结果比较表明，在非均匀序列的情况下，两种树的旋转次数都减少了10%以上，但在这种情况下高度更大。这是由于在第二种情况下，相同密钥的重复次数要比随机分布的文件多。因此，删除和插入操作没有被执行，因为在插入的情况下，值已经在树中，或者在删除的情况下，值已经被删除，这就把这些操作减少为不成功的搜索，避免了这些情况下的旋转。

表4：对密钥序列进行100,000次操作（25%插入、50%搜索和25%删除）的测量结果，该序列中的值是随机部署的。

文件名称	每次操作的平均高度		平均数量每次操作的旋转次数	
	AVL	篮板球	AVL	篮球场
R10_100000.txt	14,189	15,897	0,021	0,015
R100_100000.txt	13,325	15,813	0,023	0,018
R1000_100000.txt	11,623	13,209	0,023	0,020
R10000_100000.txt	14,356	15,359	0,029	0,028
R100000_100000.txt	14,711	14,782	0,033	0,030
RMilion_100000.txt	17,925	18,017	0,066	0,055

表5：在密钥分布概率不均匀的情况下，对密钥序列进行100,000次系列操作（25%插入，50%搜索和25%删除）的结果

文件名称	每次操作的平均高度		平均数量每次操作的旋转次数	
	AVL	篮球场	AVL	篮球场
nr10_100000.txt	15,139	15,289	0,058	0,047
nr100_100000.txt	14,682	15,046	0,042	0,035
nr1000_100000.txt	14,686	15,794	0,018	0,014
nr10000_100000.txt	15,323	16,432	0,031	0,028
nr100000_100000.txt	15,382	15,409	0,042	0,037
nrMilion_100000.txt	16,416	16,836	0,130	0,109

表4和表5显示了相同数量的操作和以相同方式建立的初始树的实验结果，与之前的实验相同。然而，这一系列操作的频率是不同的。有25%，插入，50%搜索，和25%删除。与之前的情况一样，RB和AVL树在一系列操作后的树的高度与初始树相比都没有变化。

7. 结论

对于这两种类型的树来说，序列、重复次数和键的范围对这些树的性能没有显著影响。

AVL树比RB树更有效，无论密钥分布和重复次数如何。根据这两种树的定义，可以预计，高度

在AVL树中比较低，也就是说，在AVL树的情况下，比较的数量比较小，而为了保持这种结构，旋转的数量比较大。考虑到旋转和比较（即过渡到下一级）的成本（即复杂性），以及旋转次数的不同，可以得出结论，在AVL树的情况下，这些转换导致了理想的结果。因此，从测量的一系列操作中的平均操作的价格角度来看，是合理的。对这些树有利的是，操作的实现比在RB树中更简单。

RB树虽然在搜索操作占上风时不如AVL树的效率高，但在测量参数的意义上并没有太大落后。由于RB树被用作2-3-4棵树的二进制表示，因此它们具有重要意义。正如论文[20]所显示的，在某些应用中，它们甚至比AVL树有更好的结果。

在钥匙分布不均匀的情况下，当钥匙的时间位置性增加时，就像许多应用中的情况一样，与数值随机出现的情况相比，两种树类型都不能提供改进。这是两种类型的树的不足之处，因为在这两种情况下，操作效率在很大程度上取决于初始树中的密钥部署。未来对这些树与splay树进行评估的工作很有吸引力。

8. 参考文献

[1] Tomašević M. 2011.算法和数据结构、*学术思想*，贝尔格莱德（塞尔维亚语）。

[2] Adelson-Velski G. M. 和 Landis E. M. 1962.组织信息的算法》，*《苏联数学报》。3: 1259-1263.*

[3] Živković M. 2000, Algorithms*, Matematički fakultet, Beograd.* （塞尔维亚语）。

[4] Cormen T., Leiseron Ch., Rivest R. 1990.算法简介》，*麻省理工学院出版社，McGrawHill*。

[5] Flaming B. 1993.Practical Data Structures in C++*, John Wiley and Sons.*

[6] Knuth D. 1997*,* The Art of Computer Programing*, Vol. 3, Third Edition, Addison - Wesley.*

[7] Urošević D. 1996.C编程语言中的算法、*Mikro knjiga.*(塞尔维亚语)。

[8] Aho A., Hopcroft J., Ullman J. 1983.*,* Data Structures and Algorithms*, Addison-Wesley.*

[9] Bayer R. 1971.Binary B-Trees for Virtual Memory, 1971年ACM-SIGFIDET数据描述、访问和控制研讨会论文集，加州圣地亚哥，11月11-12日。

[10] Karlton P.L., Fuller S.H., Scrogas R.E. and Kaehler E.B. 1976.高平衡树的性能，*ACM通讯，第19卷，第1期，第23-28页。*

[11] Bear J. L. and Schwab B. 1977.A Comparation of Tree-balancing Algorithms*, Comunications of the AC, 20（5）:pp.322-330.*

[12] Writght W. E. 1980.*An Empirical Evaluation of Algorithms for Dynamically Maintaining Binary Search Trees*, Proceedings of the ACM 1980 annual conference, pp.505-515.

[13] Nievergelt J. and Reingold E.M. 1973.Binary Search Trees of Bounded Balance*, SIAM Journal Computing 2, pp 33-43.*

[14] Bell J. 和 Gupta G. 1993.自我调整二进制搜索树技术的评估，*软件-实践与经验，v23。369-8.*

[15] Bitner J. R. 1979.Heuristics That *Dynamically* Organize Data Structures*, SIAM J. Computing, 8, pp 82-110.*

[16] Allen B. and Munro I. 1978., Self-organising Binary Search Trees, JACM, 25, pp 526-535.

[17] Sleator D.D.和Trajan R.E. 1985, *Self-adjusting Binary Search Trees*, Journal of the Association for Computing Machinery, Vol. 32, No. 3, July 1985, pp.652 -686.

[18] Štrbac-Savić S., Tomašević M. 2012.Analiza tehnika za reorganizaciju samopodešavajućih stabala*, In Proceedings of the Infoteh.*2012.(2012年3月21-23日。) Jahorina

[19] Pfaff B. 2004.系统软件中BST的性能分析*, ACM SIGMETRICS, pp410-411.*

[20] Neyer M. P. 2009A Comparison of Dictionary Implementations. *http://www.cs.unc.edu/~plaisted/comp750/Neyer%20paper.pdf*